Code:- Binary Search Tree

```
#include <iostream>
using namespace std;

struct node
{
   int data;
   node* left;
   node* right;
};

// Insertion
node* insert(node* root, int x)
{
   if(root == NULL)
   {
      node* p = new node();
      p->data = x;
      p->left = NULL;
      p->right = NULL;
      return p;
   }

   if(x < root->data)
   {
      root->left = insert(root->left, x);
   }
   else
   {
      root->right = insert(root->right, x);
   }

   return root;
}

// Deletion
node* delete1(node* root, int x)
{
   if(root == NULL)
   {
      return root;
   }

   if(x < root->data)
   {
      root->left = delete1(root->left, x);
   }
```

```
    else if(x > root->data)
    {
        root->right = delete1(root->right, x);
    }
    else
    {
        // No child
        if(root->left == NULL && root->right == NULL)
        {
            delete root;
            return NULL;
        }

        // Only right child
        if(root->left == NULL)
        {
            node* temp = root->right;
            delete root;
            return temp;
        }

        // Only left child
        if(root->right == NULL)
        {
            node* temp = root->left;
            delete root;
            return temp;
        }

        // Both children
        node* successor = root->right;

        while(successor->left != NULL)
        {
            successor = successor->left;
        }

        // Replace current node data
        root->data = successor->data;
        root->right = delete1(root->right, successor->data);
    }

    return root;
}

// Inorder Traversal
void inorder(node* root)
{
```

```
    if(root != NULL)
    {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

// Preorder Traversal
void preorder(node* root)
{
    if(root != NULL)
    {
        cout << root->data << " ";
        preorder(root->left);
        preorder(root->right);
    }
}

// Postorder Traversal
void postorder(node* root)
{
    if(root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}

// Main Function
int main()
{
    node* root = NULL;

    // Insertion
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    // Traversals
    cout << "Inorder Traversal: ";
    inorder(root);
```

```
    cout << "\nPreorder Traversal: ";
    preorder(root);

    cout << "\nPostorder Traversal: ";
    postorder(root);

    // Deletion of leaf node
    root = delete1(root, 20);

    cout << "\n\nAfter deleting 20:";
    cout << "\nInorder Traversal: ";
    inorder(root);

    // Deletion of root with two children
    root = delete1(root, 50);

    cout << "\n\nAfter deleting 50:";
    cout << "\nInorder Traversal: ";
    inorder(root);

    cout << endl;
    return 0;
}
```

//Output

```
Inorder Traversal: 20 30 40 50 60 70 80
Preorder Traversal: 50 30 20 40 70 60 80
Postorder Traversal: 20 40 30 60 80 70 50

After deleting 20:
Inorder Traversal: 30 40 50 60 70 80

After deleting 50:
Inorder Traversal: 30 40 60 70 80
```